

Теория:

ContentRendered – это событие, которое возникает после успешного рендеринга (отрисовки) содержимого элемента управления в WPF. Когда вы работаете с фреймом (Frame), это событие может быть использовано для выполнения каких-то действий после того, как содержимое фрейма было загружено и отрисовано.

Публичная переменная (**public**):

- Видна и доступна из любого места программы.
- Может быть использована и изменена вне класса, где объявлена.

Приватная переменная (**private**):

- Видна и доступна только внутри того класса, где объявлена.
- Недоступна и не может быть изменена извне класса.

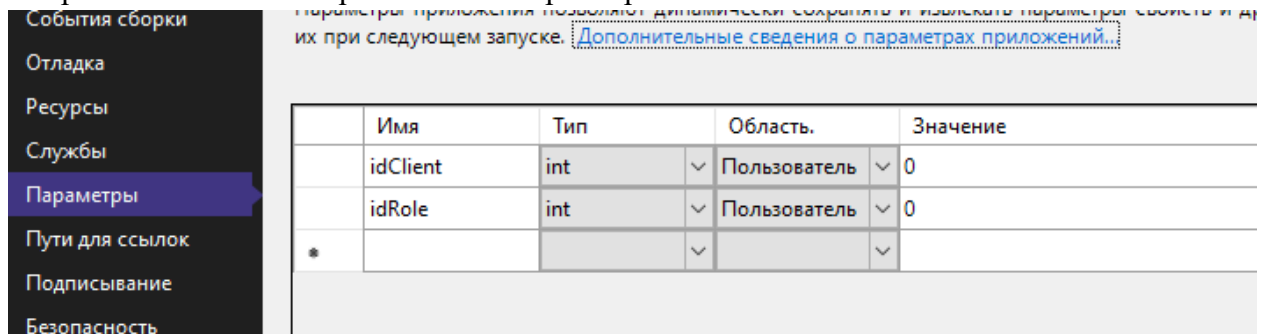
Конструкция **try...catch** используется для обработки исключений (ошибок) в коде. Она позволяет написать блок кода, который может вызвать исключение, и затем определить, как обработать это исключение.

Событие **Closed** относится к событию, которое возникает, когда окно закрывается. Это событие предоставляет возможность выполнить код после закрытия окна.

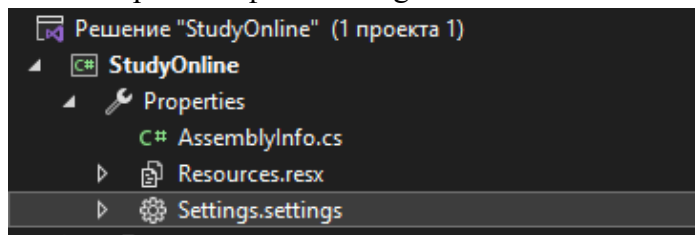
Объект класса представляет собой конкретный экземпляр (инстанс) класса. Класс определяет некоторый тип данных, а объект является переменной этого типа данных. Класс может содержать поля (переменные) и методы (функции), которые определяют состояние и поведение объекта.

Создаём новые переменные в параметрах приложения.

Открываем «Свойства проекта» → «Параметры»



Либо открываем файл Settings



Создание этих переменных поможет нам использовать их в любом месте нашего проекта.

Вернёмся к нашей кнопке «Личный кабинет». Сейчас она отображается на каждой странице, что не совсем логично, ведь она будет отображаться даже на странице авторизации.

Пропишем ей новое свойство в разметке приложения: `Visibility="Collapsed"`.

Также добавим нашему MainFrame событие ContentRendered.

В самом событии пропишем условия для отображения нашей кнопки «Личный кабинет»:

```
Ссылка: 1
private void MainFrameContentRendered(object sender, EventArgs e)
{
    if (Properties.Settings.Default.idClient == 0)
    {
        CabinetButton.Visibility = Visibility.Collapsed;
    }
    else
    {
        CabinetButton.Visibility = Visibility.Visible;
    }
}
```

Теперь поменяем страницу открытия с MainPage на AuthPage. При запуске можно увидеть, что кнопка «ЛК» пропала. Пробуем авторизоваться. Кнопка должна появиться.

Вернёмся к работе со страницей авторизации.

Во-первых, по паттерну MVVM – вся работа с БД должна реализовываться в папке ViewModel.

Создадим в данной папке класс UsersVM (UsersViewModel) для работы с классом (таблицей в БД) Users.

В созданном классе нам необходимо создать публичную переменную, которая на вход будет принимать строки: логин и пароль.

```
public bool CheckAuth(string login, string password)
```

Для удобства пользования приложением надо учитывать разные варианты развития событий, при которых может возникнуть ошибка. Поэтому сначала проверяем, что в качестве логина нам пришла не пустая строка.

На странице авторизации мы будем использовать конструкцию try...catch, поэтому при проверке данных мы будем использовать возврат исключения:

```
throw new Exception("Вы не ввели логин.");
```

По аналогии с логином необходимо сделать проверку на ввод пароля.

Если эти проверки успешно пройдены, программа должна приступить к поиску данных в БД. Самый простой способ – создать переменную для подсчета количества найденных строк, в которых логин и пароль соответствуют введенным.

Затем прописываем возвращаемое исключение в случае ошибки, и, так как у нас переменная bool, то при успехе возвращаем «true»:

```
if (checkClient == 0)
{
    throw new Exception("Пользователь не найден.");
}
else
{
    return true;
}
```

Наш класс на поиск логина и пароля готов. Переходим к странице авторизации.

В событии клика необходимо создать конструкцию try...catch, либо брать уже готовый код в неё.

Внутри «try» необходимо создать объект нашего класса из ViewModel:

```
UsersVM newObject = new UsersVM();
```

Не забываем подключить using к нашей папке ViewModel.

Создаем проверку, условием в которой будет отсутствие пустоты и пробелов в наших полях для ввода.

Внутри оператора if создаём переменную для проверки наличия данных в БД, то есть для работы с нашей переменной из UsersVM:

```
bool result = newObject.CheckAuth(LoginTextBox.Text, AuthPasswordBox.Password);
```

Если проверка прошла успешно и нам вернулось «true», то приступаем к работе с переменными из параметров приложения.

Для начала нашему списку из пользователей присваиваем значение, в котором должны быть строки с такими же логином и паролем. По логике, такая строка должна быть одна.

Затем, перебирая каждый элемент нашего списка, присваиваем нашим переменным idUser и idRole значения и сохраняем их:

```
Properties.Settings.Default.idClient = item.IdUser;  
Properties.Settings.Default.idRole = item.IdRole;  
Properties.Settings.Default.Save();
```

После перебора списка должен произойти переход на главную страницу.

НО: если строка с такими логином и паролем не была найдена, необходимо сообщить пользователю, что такого пользователя не существует. А если в полях ввода окажется пустота или пробелы, необходимо сообщить, что данные не введены.

Теперь пробуем запустить проект и авторизоваться снова.

При успехе мы попадем на главную страницу и у нас начнет отображаться кнопка «Личный кабинет». Закрываем проект и открываем снова. Сначала по-прежнему открывается окно авторизации, но кнопка осталась на месте. Будем исправлять.

Для этого возвращаемся в верстку главного окна и добавляем ему событие закрытия окна.

В самом событии приравниваем переменные к нулю и сохраняем:

```
private void WindowClosed(object sender, EventArgs e)  
{  
    Properties.Settings.Default.idClient = 0;  
    Properties.Settings.Default.idRole = 0;  
    Properties.Settings.Default.Save();  
}
```

Проверяем итог.